How to Run the Prototype

eBPF bufferbloat detection + adaptive AQM | CSC 303, IIT Roorkee

1. What the prototype does

It closes the loop **measure -> detect -> decide -> adapt -> verify** on a three-namespace testbed with one 10 Mbit bottleneck. An eBPF program measures how long each packet waits in the router's queue; a Python controller reads that every 0.5 s and, when it sees a standing queue, swaps the queue discipline (qdisc) along the ladder `pfifo -> fq_codel -> cake`. When the load goes away it goes back to `pfifo`.

```
bb-h1 ------- bb-r ------- bb-h2        r-eth1 : htb 10 Mbit + leaf qdisc (managed)
 ping,        r-eth1                    h2-eth0: netem 20 ms (baseline RTT)
 iperf3 client  ^                       iperf3 server
                +-- eBPF observer + controller
```

2. Files

File	Role
testbed.sh	Creates/destroys the namespaces, veth links, 10 Mbit shaper with a 1000-packet pfifo, and 20 ms path delay. `up | down | status`
observer.bpf.c	eBPF on tracepoints `net:net_dev_queue` and `net:net_dev_start_xmit`: per-packet queue sojourn time, aggregated per 100 ms; per-flow byte counters.
controller.py	Reads the eBPF maps, qdisc stats and a ping probe twice a second; runs the detector and state machine; replaces the qdisc with `tc`.
run_demo.sh	Runs the same scripted load twice: controller off (control), then on (adaptive).
plot.py	Prints the percentile table and writes `results/demo.png`.
setup.sh	One-time dependency install on a fresh Debian/Ubuntu machine.

3. Requirements

Linux only: qdiscs, network namespaces and eBPF are Linux kernel features. On the Mac the Linux side is a **Lima VM named** `bloat` (Debian 12, kernel 6.1, BCC 0.26). The project folder is mounted inside the VM at the *same path*, so files written in the VM appear on the Mac immediately. Plotting runs on the Mac (matplotlib is installed there, not in the VM).

4. Before the demo (checklist)

```
cd ~/Desktop/PROTOTYPE
limactl list                # expect:  bloat   Running
limactl start bloat         # only if it shows Stopped (about 30 s)
```

- Make the terminal font large and the window wide - the live table is about 120 columns.
- Keep `results/demo.png` open as a backup in case anything fails live.

5. Option A - scripted demo (recommended, about 2 minutes)

```
# both runs: static pfifo (controller off), then adaptive  (~2 min)
limactl shell --workdir "$PWD" bloat sudo ./run_demo.sh

# or only the adaptive run  (~1 min)
limactl shell --workdir "$PWD" bloat sudo ./run_demo.sh adaptive

# back on the Mac: summary table + plot
python3 plot.py results
open results/demo.png
```

Each run is: 8 s idle, 30 s of bulk TCP from h1 to h2 (iperf3), 14 s idle. The ping probe runs the whole time. Logs are written to `results/` (`*.csv`, `*.ping.csv`, `*.iperf.json`).

6. Option B - drive it by hand (live)

Open two Mac terminals. In each, enter the VM and the project folder:

```
limactl shell bloat
cd ~/Desktop/PROTOTYPE
```

Terminal 1 - testbed + controller

```
sudo ./testbed.sh up
sudo python3 controller.py          # live table; Ctrl-C to stop
```

Terminal 2 - generate load

```
sudo ip netns exec bb-h2 iperf3 -s -D                    # server (daemon)
sudo ip netns exec bb-h1 iperf3 -c 10.0.2.2 -t 30        # 30 s bulk TCP

# show the qdisc being swapped, live:
sudo ip netns exec bb-r tc qdisc show dev r-eth1
```

Clean up

```
sudo ./testbed.sh down
```

Useful controller options: `--observe-only` (detect but never act), `--duration N`, `--enter-ms` / `--exit-ms` (thresholds, default 50 / 20), `--persist` (default 2 s), `--no-revert`, `--no-ebpf` (fallback delay from backlog/rate). Full list: `python3 controller.py -h`.

7. What to point at while it runs

Column	Meaning
state	NORMAL, DEGRADED, MITIGATING, RECOVERED
qdisc	the leaf qdisc currently on the bottleneck
rate / backlog	egress rate and bytes queued
sojourn	eBPF: minimum queue wait in the last 100 ms window
rtt / infl	ping probe RTT and its inflation over its own minimum
top flow	heaviest flow from the eBPF per-flow map

Control run (pfifo, controller off): RTT climbs to about 1 second - that is bufferbloat.

Adaptive run: the >>> lines show each decision. From a real run:

```
 7.6 s  state NORMAL -> DEGRADED     (standing queue 2 s: 171 ms, link saturated)
 8.1 s  qdisc pfifo -> fq_codel      (escalate); DEGRADED -> MITIGATING
14.2 s  state MITIGATING -> RECOVERED  (delay 136 -> 0 ms)
42.5 s  qdisc fq_codel -> pfifo     (link idle for 8s, back to base)
```

8. How it decides

A standing queue is declared only when **all three** hold for 2 s:

1. the link is saturated (egress ≥ 85% of capacity);
2. the minimum sojourn time in every 100 ms window is above 50 ms (every packet waited);
3. the ping RTT is inflated by more than 50 ms over its own minimum.

```
NORMAL --standing queue 2 s--> DEGRADED --next rung--> MITIGATING --6 s dwell--> RECOVERED
  ^                                                    | no better: escalate         |
  |                                                    | worse: roll back            |
  +--------------- link idle 8 s: back to pfifo <-------+-----------------------------+
```

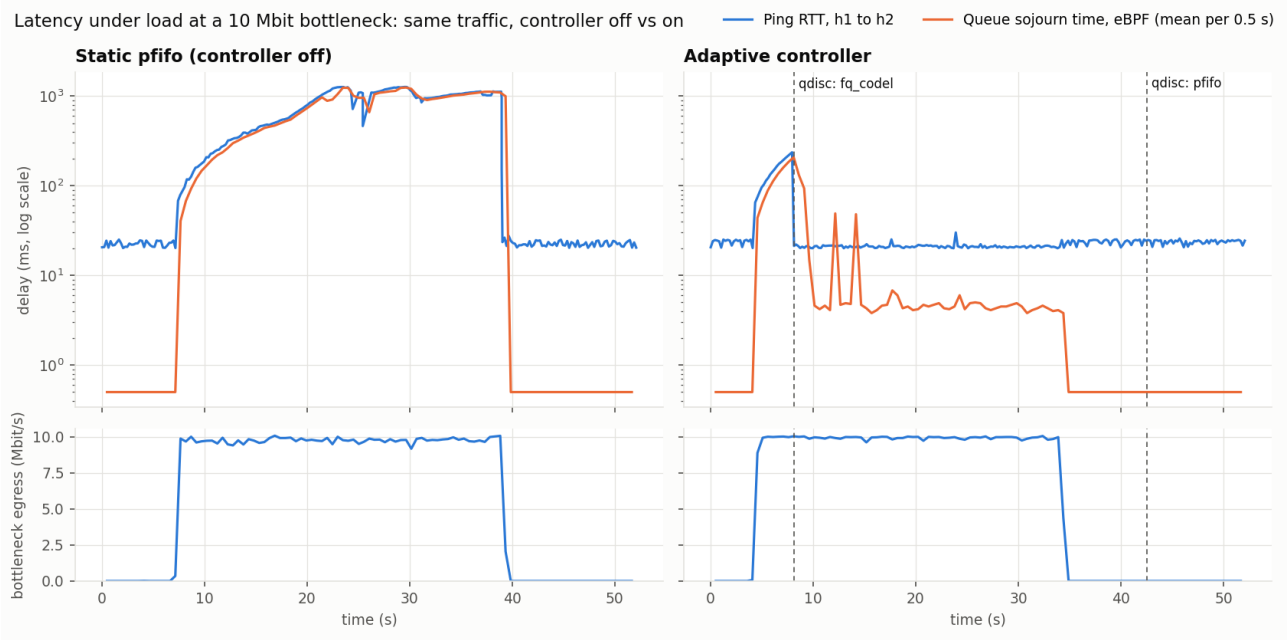
It reverts only when the load is gone, never because delay is low - low delay is the *effect* of the fix, so reverting on it would flap.

9. Expected result

Ping probe RTT while the bulk flow is running (ms), and what it cost (latest run):

Run	p50	p95	p99	Goodput	Retrans	Drops	Switches
Static pfifo (controller off)	953.5	1252.0	1262.0	9.32 M	341	343	0
Adaptive controller	21.1	163.2	219.7	9.47 M	281	88	2

Median latency under load drops from about 950 ms to about 21 ms with no loss of throughput. The adaptive p95/p99 come from the ~3 s between the load starting and the qdisc switch.



results/demo.png

10. Troubleshooting

Problem	Fix
"File exists" / namespace already exists	A previous run did not clean up: `limactl shell bloat sudo ./testbed.sh down`, then retry.
"limactl: command not found"	You are already inside the VM (prompt shows `@lima-bloat`). Drop the limactl part: just `sudo ./run_demo.sh`. Type `exit` to return to the Mac for plotting.
"run with sudo"	Prefix the command with `sudo` (inside the VM).
VM is stopped	`limactl start bloat`
"matplotlib is not installed here"	Expected inside the VM - run `python3 plot.py results` on the Mac.
Check testbed state	`sudo ./testbed.sh status` (inside the VM)
VM will not start at all	Show the saved `results/demo.png` and `results/summary.txt`.

11. Fresh Linux machine (no Lima)

```
./setup.sh                # once: iproute2, iperf3, BCC, kernel headers, qdisc modules
sudo ./run_demo.sh        # both runs
pip install matplotlib && python3 plot.py results
```

12. Known limitations

• One run per condition, not repeated trials.
• No comparison yet against static fq_codel or static cake.
• cake was never reached (fq_codel fixed this scenario), so escalate/rollback are untested.
• Only qdisc switching, no parameter tuning (target / interval).
• The ping probe is a sparse flow and gets priority under fq_codel, so it understates harm; the bulk flow's own sojourn stays at ~5 ms.
• Thresholds are illustrative; observer overhead is not measured.
• No fairness, multi-flow, varying-capacity or UDP-flood scenarios.